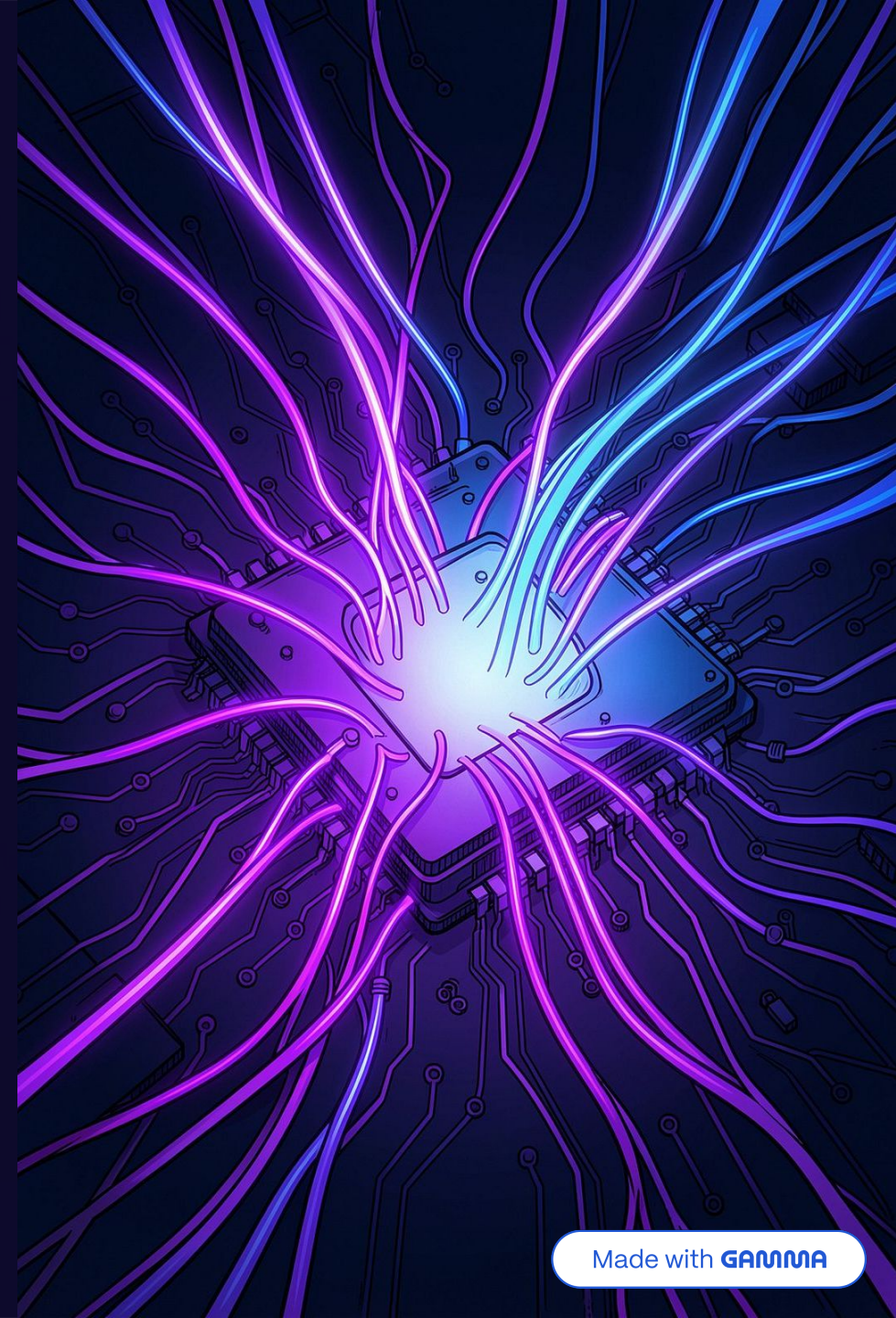


ARQUITECTURA DE SOFTWARE II · CLASE PRÁCTICA

Concurrencia y Paralelismo en Go

Después de Caché, Solr y RabbitMQ, llegamos al núcleo: **cómo un mismo proceso hace varias cosas a la vez** de forma elegante y segura.



Diccionario de Términos

Conceptos clave que dominaremos hoy:

Goroutine	Función liviana que corre de forma independiente
Channel	Tubería para comunicar goroutines
Select	Espera sobre varios channels a la vez
WaitGroup	Espera a que terminen N goroutines
Mutex	Candado que protege datos compartidos
Race Condition	Dos goroutines escribiendo el mismo dato sin control
Worker Pool	N goroutines fijas procesando trabajos de una cola
Context	Cancela o pone timeout a una operación
Fan-out / Fan-in	Repartir trabajo entre goroutines y juntar resultados

El Problema: Latencia Acumulada

Para armar la ficha de un producto y luego actualizarla necesitamos consultar tres fuentes independientes. En serie, las latencias se suman:

 Precio

Base de datos

 Stock

Servicio externo

 Reviews

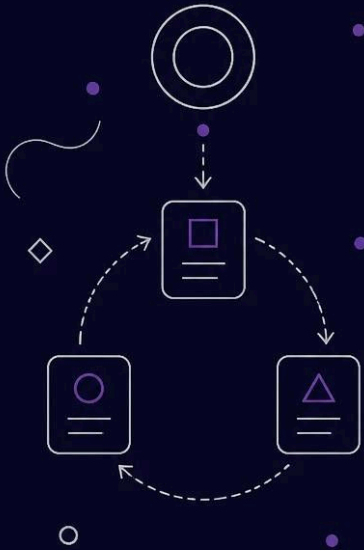
API de terceros

Actualizar 10.000 productos uno por uno es inviable. Necesitamos ejecutar tareas independientes **en simultáneo** sin romper la consistencia.

Concurrencia vs. Paralelismo

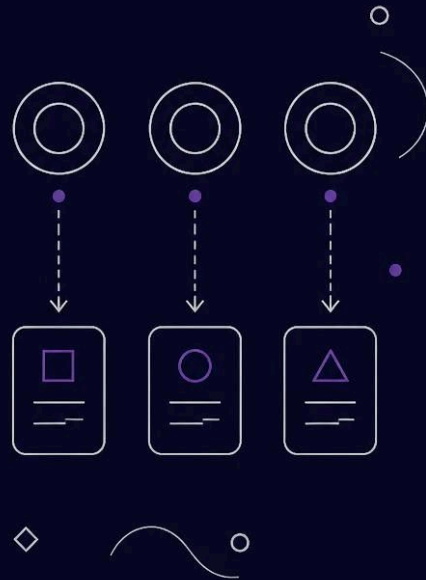
Concurrencia

(diseño del software,
tareas entrelazadas
en un solo núcleo,
un cocinero rotando
entre tres ollas,
depende del programador)



Paralelismo

(hardware,
tareas físicamente
simultáneas en múltiples núcleos,
tres cocineros cada uno
con su olla,
depende de los cores disponibles)



¿Cuál es la diferencia real?

Concurrencia es estructurar el problema en tareas independientes que pueden avanzar entrelazadas. Depende del *diseño*.

Paralelismo es que esas tareas se ejecuten físicamente al mismo tiempo. Depende del *hardware*.

Go te permite programar de forma concurrente; el runtime decide cuánto paraleliza según los núcleos disponibles.

Goroutines: Hilos Ultraligeros

¿Cómo se lanzan?

Con la palabra clave `go` delante de cualquier función. Arrancan con ~2 KB de stack y el runtime multiplexa **miles** sobre pocos threads del SO.

```
go buscarPrecio(productoID)
go actualizarStock(id, qty)
go enviarEmail(usuario)
```

- ❏ El runtime de Go multiplexa miles de goroutines sobre un puñado de threads reales del sistema operativo.



Channels: Comunicación sin Memoria Compartida

Los channels son tuberías tipadas que conectan goroutines. **No compartas memoria para comunicarte; comunícate para compartir memoria.**

Channel sin buffer (sincrónico)

```
ch := make(chan int)
// envía y bloquea hasta recepción
ch <- 42
val := <-ch
```

Channel con buffer (asincrónico)

```
ch := make(chan int, 3)
// envía sin bloquear (hasta 3)
ch <- 1
ch <- 2
ch <- 3
```

Select: Espera sobre Múltiples Channels

¿Qué hace `select`?

Permite que una goroutine espere sobre varios channels y reaccione al primero que tenga datos. Si ninguno está listo y hay un `default`, lo ejecuta; si no, queda bloqueada.

- ❏ Caso de uso típico: recibir el resultado que llegue primero entre "precio desde caché" o "precio desde base de datos".

```
select {  
  case precio := <-cacheCh:  
    fmt.Println("Caché:", precio)  
  case precio := <-dbCh:  
    fmt.Println("BD:", precio)  
  case <-time.After(100 * time.Millisecond):  
    fmt.Println("Timeout")  
}
```

Race Conditions y Mutex

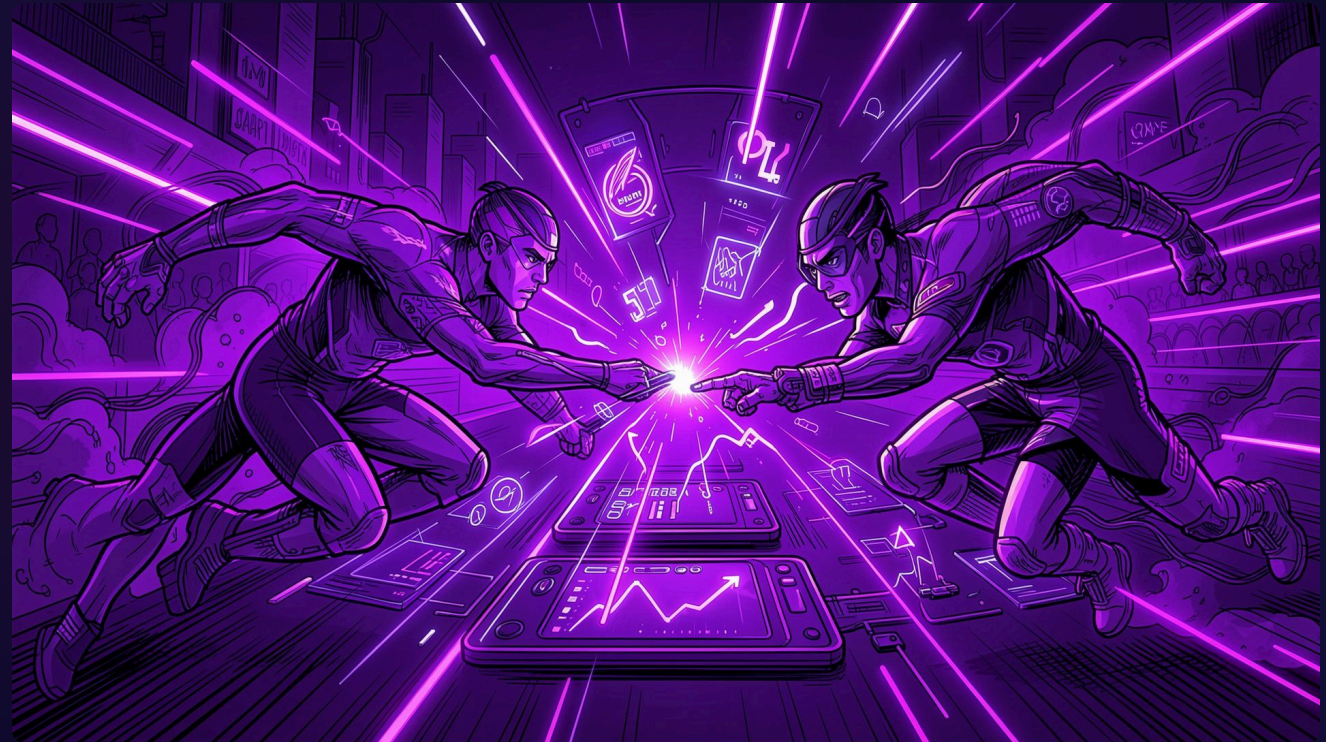
El peligro: escritura concurrente

Si dos goroutines escriben el stock del mismo producto sin protección, el resultado es **impredecible**.

```
var mu sync.Mutex
var stock int

mu.Lock()
stock -= 1 // sección crítica
mu.Unlock()
```

⚠ Detecta races con: `go run -race main.go`



WaitGroup: Sincronización de Goroutines

Cuando lanzamos varias goroutines y necesitamos esperar a que **todas terminen** antes de seguir, usamos `sync.WaitGroup`.

```
var wg sync.WaitGroup

for _, id := range productos {
    wg.Add(1)
    go func(pid int) {
        defer wg.Done()
        actualizar(pid)
    }(id)
}

wg.Wait() // bloquea hasta cero
fmt.Println("Todos listos")
```

Los tres métodos

Add(n)

Registra N goroutines pendientes

Done()

Señala que una terminó (llamar con `defer`)

Wait()

Bloquea hasta que el contador llegue a cero

Patrón Fan-Out / Fan-In

Se reparte el trabajo entre varias goroutines (**fan-out**) y se recolectan los resultados en un único channel (**fan-in**). Es la forma natural en Go de consultar fuentes en paralelo.



❏ Este patrón resuelve directamente el problema del catálogo: consultar precio, stock y reviews en paralelo y consolidar la ficha del producto sin sumar latencias.

Worker Pool: control sobre las goroutines

Lanzar una goroutine por tarea puede saturar recursos. El **Worker Pool** crea un número fijo de workers que consumen de un channel compartido.

Caso práctico: actualizar precios de miles de productos sin descontrolar goroutines activas.

⚠ Sin pool: 10.000 productos = 10.000 goroutines simultáneas. Con pool de 50 workers: solo 50 activas.



Context: timeouts y cancelación

`context.WithTimeout` propaga cancelación entre goroutines. Si un servicio externo no responde, seguimos sin esperarlo.

Código: timeout de 300ms

```
ctx, cancel := context.WithTimeout(
    context.Background(),
    300*time.Millisecond,
)
defer cancel()

reviews, err := fetchReviews(ctx, productID)
// Si tarda >300ms → ctx.Done() → error
```

¿Por qué importa?

Una goroutine bloqueada para siempre es un **goroutine leak**.
Context garantiza que toda la cadena se cancele limpiamente.



`defer cancel()` siempre: libera recursos aunque la operación termine antes.

Goroutine Leaks y Deadlocks

El problema del channel sin lector

```
ch := make(chan int)
go func() {
    ch <- 42 // bloquea para siempre
}()
// nadie lee → leak garantizado
```

La goroutine queda bloqueada eternamente si nadie lee del channel. Context con timeout es la red de seguridad.

Reglas de oro

Cierra el channel quien escribe

Nunca quien lee — causa panic.

Usa Context siempre

Propaga cancelación a toda la cadena.

Testea con `-race`

Detecta accesos no protegidos.

Reproducir una Race Condition

Código deliberadamente incorrecto

```
var stock int = 100

for i := 0; i < 1000; i++ {
    go func() {
        stock-- // ← data race
    }()
}

fmt.Println(stock) // ¿cuánto vale?
```

1000 goroutines modifican `stock` sin sincronización. El resultado es **impredecible**.

Pasos para reproducir

01

Guardar el código en `race.go`

02

Ejecutar con el detector

```
go run -race race.go
```

03

Leer el reporte

Go muestra las dos goroutines en conflicto y la línea exacta.

Corregir con `sync.Mutex`

Solución con Mutex

```
var (  
    stock int = 100  
    mu sync.Mutex  
)  
  
for i := 0; i < 1000; i++ {  
    go func() {  
        mu.Lock()  
        stock--  
        mu.Unlock()  
    }()  
}
```

El Mutex garantiza acceso **excluyente** a la variable compartida.

Verificación

Correr nuevamente con `go run -race`: el reporte desaparece y el resultado es predecible.



 Alternativa: un channel como semáforo con capacidad 1. Comparar ambas soluciones en el Ejercicio 4.

Patrones en acción

1

Fan-Out / Fan-In: ficha de producto

Consultar **precio**, **stock** y **reviews** en paralelo y armar la respuesta combinando los tres resultados.

2

Worker Pool: actualización masiva de precios

Actualizar el precio de una lista grande de productos usando un pool de workers fijo en vez de una goroutine por producto.

Timeout y comparación de estrategias

1

Context: cortar una fuente lenta

Agregar un timeout a una de las fuentes del Ejercicio 1 para que no bloquee la respuesta si tarda demasiado.

2

★ Bonus: Contador seguro — Mutex vs Channel

Resolver el mismo contador de stock concurrente de dos formas distintas y comparar legibilidad y comportamiento.

  Los enunciados completos con dominio, datos de entrada y criterios de entrega están en la guía impresa.